# Methodology and Architecture Assessment

## Hybrid Thesis Topic Novelty Assessment Model

Assignment 2  Project methodology and architectural approach

Alisher Tokesh  CSE 2507M

Project basis: novelty-similarity-detection-model repository

October 2026

## Table of Contents

1. Project Analysis 3
1.1 Goals and development context 3
1.2 Requirements and priorities 4
1.3 Constraints and risks 4
2. Development Methodology Assessment 5
2.1 Recommended hybrid approach 5
2.2 Fit, risks and alternatives 6
3. Architectural Approach Assessment 7
3.1 Current state and target design 7
3.2 Quality attributes and adjustments 8
4. Conclusions and Recommendations 9
References 9

## Executive Position

This report evaluates the existing Hybrid Thesis Topic Novelty Assessment Model as a practical module for an intelligent thesis-management platform. The repository already provides an explainable hybrid score, interchangeable embedding backends, a command-line and FastAPI interface, automated tests, and a PostgreSQL plus pgvector schema. The principal recommendation is a modular monolith delivered through four evidence-based increments. This approach preserves flexibility for threshold calibration and multilingual data while keeping governance, auditability, and operational cost proportionate to the approved pilot case.

**Planning baseline from Assignment 1.** The methodology and architecture are assessed against the 17-week stage-gated R and D implementation plan, a direct-cost estimate of $38,188, and a 15 percent contingency reserve of $5,728, for a total planned budget of $43,916. The model is decision support: it flags high-risk similarity cases, while academic approval remains a human decision.

# 1. Project Analysis

## 1.1 Goals and Development Context

The product objective is to help a university identify duplicate or near-duplicate thesis topics before approval and to provide a transparent basis for reviewer judgement. A submitted title, abstract, keywords and domain are normalized, embedded, compared with an existing corpus, and evaluated for keyword uniqueness and domain rarity. The result is a novelty score, duplicate probability, label, explanation components, and the most similar historical topics.

**Primary goals.** Reduce duplicate-topic risk; support multilingual academic topics; make a recommendation explainable rather than purely opaque; preserve a reproducible baseline for research; and expose the assessment through an API that can later join a wider thesis-management platform.

**Current implementation evidence.** The model calculates semantic distance, keyword uniqueness, and domain rarity with configurable weights (0.60, 0.25, 0.15). Default similarity thresholds classify duplicate at 0.90 or above and near-duplicate at 0.75 or above. The repository offers a deterministic hashing fallback for tests and an optional multilingual SentenceTransformer backend for production-like evaluation.

### Development Context

| Factor | Assessment | Implication |
|---|---|---|
| Project type | Applied ML and API module | Both model quality and software quality must be demonstrated. |
| Users | Academic administrators and supervisors | Explainability and reviewer control are essential. |
| Data | Historical topic corpus in several languages | Data quality and labels are major delivery risks. |
| Lifecycle | Pilot, then platform integration | Avoid irreversible coupling to a UI or one model vendor. |

*Table 1. Context derived from the repository and the stated thesis-platform purpose*

## 1.2 Requirements and Priorities

Priorities use MoSCoW: Must means required for a trustworthy pilot; Should means high value after the vertical slice works; Could is deliberately deferred. Quantitative non-functional targets are pilot acceptance targets proposed for planning.

| Type | Priority | Requirement | Acceptance signal |
|---|---|---|---|
| Functional | Must | Assess a topic against a corpus and return top similar topics, score, probability and label. | Deterministic API response with top-k evidence. |
| Functional | Must | Ingest existing topics and keep corpus embeddings consistent with the chosen model. | Rebuild process records corpus and model version. |
| Functional | Should | Allow a reviewer to confirm or override the suggested label with a reason. | Decision and reason are auditable. |
| Functional | Should | Evaluate thresholds on labelled duplicate, near-duplicate and novel pairs. | Precision, recall and F1 report. |
| Functional | Could | Provide a separate administrative dashboard and retraining automation. | Deferred beyond pilot unless demand is proven. |
| Nonfunctional | Must | Explainability and reproducibility of every assessment. | Store score components, thresholds and model ID. |
| Nonfunctional | Must | Privacy and access control for topics and reviewer decisions. | RBAC, encrypted transport, restricted logs. |
| Nonfunctional | Should | p95 assessment response under 1.5 seconds for a 10,000-topic pilot corpus. | Load test using production-like embeddings. |

*Table 2. Prioritized requirements*

## 1.3 Constraints and Risks

| Constraint or risk | Why it matters | Control |
|---|---|---|
| 17-week schedule and $43,916 cap | There is limited room for rework or infrastructure complexity. | Critical-path control, vertical slices and protected contingency. |
| Sparse or inconsistent historical records | Similarity can look precise while being wrong. | Data profiling, sampling and a labelled evaluation set. |
| Multilingual semantic variation | A fallback vectorizer is insufficient for final claims. | Evaluate multilingual E5 against a frozen benchmark. |
| False duplicate decision | Can unfairly block a student's topic. | Human-in-the-loop decision, threshold review and audit trail. |

# 2. Development Methodology Assessment

## 2.1 Recommended Hybrid Approach

The recommended methodology is iterative incremental delivery within the 17-week stage-gated R and D plan: Scrum-style two-week planning and review cycles for product and model work, Kanban limits for data remediation and review tasks, and evidence gates at the key critical-path transitions. The gates are not a second management system. They are evidence checks because a classifier can be technically deployable while still failing its educational or fairness purpose.




*Figure 2. Four short delivery increments with explicit evidence gates*

**Increment 0 weeks 1 to 3.** Complete initiation, requirements, data-access approval, privacy classification, duplicate definitions, labelled-pair protocol, baseline metrics, and a definition of done. Gate 1 requires approved data access and traceable corpus provenance.

**Increment 1 weeks 4 to 6.** Deliver corpus preparation, a hashing baseline, model adaptation, explainable result contract, and test fixtures. The critical path passes from data approval through baseline adaptation into backend integration.

**Increment 2 weeks 7 to 11.** Add multilingual E5 embeddings, pgvector retrieval, threshold evaluation, API integration, coordinator workflow, reviewer override, and a simple audit event. Gate 2 checks F1 by class, error samples, and latency before QA begins.

**Increment 3 weeks 12 to 17.** Complete QA, security and performance testing; run the coordinator pilot; collect disagreement reasons; then deploy, train users, and close the project. Gate 3 requires documented residual risks and user acceptance before rollout.

### Roles and Cadence

| Role | Responsibility | Cadence |
|---|---|---|
| Project manager and academic lead | Own schedule, budget, policy, examples and reviewer workflow. | Biweekly review |
| Data engineer | Corpus quality, access controls and labelled validation pairs. | Weekly data review |
| ML engineer | Embeddings, benchmark, threshold calibration. | Daily work and experiment review |
| Backend engineer | API, persistence, security and observability. | Daily work and integration review |
| Domain reviewers and QA | Labels, challenge cases and acceptance testing. | Weekly sample review |

## 2.2 Fit, Risks and Alternatives

This hybrid approach fits the project because the algorithm's thresholds, corpus quality and acceptable trade-offs cannot be reliably specified at the start. Small increments make technical and policy assumptions visible early, while the Assignment 1 critical path keeps work focused on data approval, baseline adaptation, API and pgvector integration, workflow, QA, pilot, deployment and closure. A full Scrum implementation would be excessive for a cross-functional part-time team, so the recommended version retains only useful ceremonies: a short planning session, visible work board, weekly quality review, and demonstrable increment every two weeks.

| Criterion | Hybrid incremental | Waterfall | Kanban only |
|---|---|---|---|
| Model and data uncertainty | 5 learn and recalibrate | 2 late feedback | 4 flexible, weaker checkpoints |
| Fixed pilot deadline | 4 time-boxed slices | 4 predictable plan | 3 flow can obscure deadline |
| Small team overhead | 4 lightweight roles | 4 low ceremony | 5 very low ceremony |
| Governance and audit | 5 gates plus evidence | 5 document-oriented | 2 needs added controls |
| Adaptability after pilot | 5 backlog and adapters | 2 change requests | 4 continuous flow |

*Table 3. Relative fit scored 1 low to 5 high*

## Methodology Risks and Responses

| Risk | Mitigation | Alternative trigger |
|---|---|---|
| Iterations turn into unmeasured experimentation | Freeze benchmark; log dataset, model and threshold version; use gate criteria. | Use a short research protocol if experiments exceed one sprint. |
| Stakeholders cannot attend reviews | Asynchronous demo, decision log, named delegate. | Adopt a fixed weekly approval window. |
| Kanban data tasks grow without bound | WIP limit and explicit backlog triage. | Split data remediation into a separately funded stream. |
| Institution requires fixed compliance sign-off | Move threat model and data approval into Increment 0. | Use a lightweight Waterfall phase only for compliance artifacts. |

**Decision.** A pure Waterfall plan is acceptable only where the data, acceptance rules and legal controls are already stable. A pure Kanban system is preferable later for operations and corpus maintenance, but it is not sufficient alone for the first evidence-based release.

# 3. Architectural Approach Assessment

## 3.1 Current State and Target Design

The implemented code is already close to a modular monolith: preprocessing, embeddings, keyword analysis, schemas, scoring model, CSV I/O, CLI and FastAPI are separate modules. The EmbeddingBackend protocol lets the system switch between hashing and SentenceTransformer encoders. The production persistence path is not yet wired into the model: the running prototype stores topics and vectors in process memory, although the repository includes a pgvector schema with a cosine HNSW index.

**Recommended style.** Keep a deployable modular monolith, but formalize it as a hexagonal design. The assessment application service owns the use case; domain scoring remains independent of infrastructure; adapters implement the API, embedding provider, repository, audit logging and observability. This gives the pilot clear boundaries without the operational burden of microservices.

| Platform or reviewer UI | FastAPI adapter | Assessment application service | Domain scorer and policy |
|---|---|---|---|
| Integration API | Authentication and audit adapter | Embedding provider port | Topic repository port |
| Reviewer decision | Metrics and logs | Multilingual E5 model | PostgreSQL plus pgvector |

*Figure 1. Target modular monolith and its replaceable adapters*

## Why This Architecture Fits

| Need | Alignment | Limit or decision |
|---|---|---|
| Top-k similarity at scale | pgvector adapter and existing HNSW schema support indexed cosine search. | Validate recall and latency before selecting index settings. |
| Explainable result | Scorer returns components, similar topics and label separately from transport. | Persist configuration and reviewer decision too. |
| Model replacement | EmbeddingBackend protocol isolates encoder choice. | Store embedding dimension and model version; re-index on change. |
| Small-team delivery | One deployable service simplifies releases and monitoring. | Do not split services until load or team boundaries justify it. |

### 3.2 Quality Attributes and Adjustments

| Attribute | Current support | Required adjustment |
|---|---|---|
| Reliability | Unit tests cover duplicate and unrelated cases; API has health endpoint. | Add integration tests for storage, restart recovery, timeouts and malformed corpus data. |
| Security and privacy | No authentication or audit layer is visible in current API. | Add platform RBAC, TLS, least-privilege database role, redacted logs and retention policy. |
| Performance | In-memory ranking is simple; schema contains HNSW index. | Use pgvector retrieval, batch embedding, bounded top-k and measured load tests. |
| Maintainability | Separate modules and typed dataclasses aid change. | Introduce repository and audit ports; document ADRs and contract versioning. |
| Reproducibility | Configurable weights and deterministic fallback exist. | Version datasets, embeddings, thresholds and evaluation reports together. |

*Table 4. Quality attribute assessment*

## Justified Architectural Adjustments

**1. Persistence boundary.** Replace direct list ownership in the service deployment with a TopicRepository port and a pgvector implementation. It should return candidate records and vector-search metadata; scoring remains in the application/domain layer.

**2. Decision audit.** Persist assessment ID, corpus version, embedding model, threshold and weight configuration, components, returned candidates, reviewer decision and reason. This enables appeals, calibration analysis and reproducible research claims.

**3. Controlled asynchronous work.** Ingestion, embedding rebuilds and evaluations should run as jobs rather than inside a request. The assessment path stays synchronous and bounded; rebuild failures cannot interrupt normal review.

**4. Observability and policy.** Record latency, error rate, label distribution, reviewer disagreement and drift in similarity distributions. Logs must avoid storing raw abstracts unless the approved data policy permits it.

**5. Avoid premature microservices.** Split the embedding or evaluation worker later only when independent scaling, separate deployment cadence, or security isolation produces a measured benefit. For the pilot, a single service with explicit ports is the lower-risk choice.

# 4. Conclusions and Recommendations

The recommended methodology and architecture are well aligned with the project goals. The hybrid score and multilingual retrieval problem require learning cycles, but the effect on students and academic approval means those cycles need visible evidence, stable evaluation data and human review. A compact iterative process with quality gates therefore offers a better fit than a rigid linear plan or unstructured continuous flow.

A modular monolith with hexagonal boundaries also matches the present codebase and the likely pilot constraints. It uses existing seams around embeddings and API delivery, adopts the repository's pgvector direction for scalable search, and reserves microservice extraction for a demonstrated need. The strongest missing capabilities are not another scoring formula; they are persistent corpus handling, policy-controlled auditability, security controls, benchmark governance and production observability.

## Prioritized Recommendations

| When | Recommendation | Expected result |
|---|---|---|
| Weeks 1 to 3 | Approve data access, define duplicate policy, freeze labelled benchmark and metric thresholds. | A governable baseline and an executable critical-path start. |
| Weeks 4 to 6 | Prepare the corpus, adapt the baseline model and document threshold evidence. | Reproducible model behaviour before integration. |
| Weeks 7 to 11 | Deliver pgvector adapter, API, coordinator workflow, audit event and measurable retrieval. | Production-like assessment on the integration critical path. |
| Weeks 12 to 17 | Complete QA and security review, pilot, deployment, training and closure within the $43,916 baseline. | Evidence for scale-up or targeted revision. |

## References


1. Hybrid Thesis Topic Novelty Assessment Model. README.md. Repository documentation, accessed October 2026.
2. thesis_novelty.model, config, embeddings, api and schemas modules. Source code reviewed for this report.
3. sql/pgvector_schema.sql. Proposed PostgreSQL and pgvector persistence schema reviewed for this report.
4. Assignment 2 brief supplied by the course. Planning assumptions in this report were created solely for the academic case study.